Assignment No:1.Set: A (1)

Assignment Name: Implement the C Program to create a child process using fork(), display parent and child process id. Child process will display the message "I am Child Process" and the parent process should display "I am Parent Process".

```
#include<stdio.h>

#include<sys/types.h>

#include<unistd.h> pid_t

pid, mypid, myppid;

void ChildProcess(); /* child process prototype */ void

{

ParentProcess(); /* parent process prototype */ int

main()

pid_t pid,mypid, myppid;

pid=getpid();

pid = fork(); if

(pid == 0)

ChildProcess();

}

{

else

ParentProcess();

return 0;

void ChildProcess()

mypid=getpid();

/**************

myppid=getppid();

printf("\n\nl am child process.. my id is %d & parent processid=%d", mypid, myppid);
```

```
}
void ParentProcess()
{
pid_t mypid, myppid;
}
sleep(1); mypid=getpid(); myppid=getppid();
printf(“\n\nl am parent process.. my id is %d and %d",mypid, myppid);
```

OUTPUT

I am child process.. my id is 5974 & parent processid=5973

I am parent process.. my id is 5973 and 5943

Assignment No:1 Set: A (2)

Assignment Name: Write a program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice() system call.

```
#include<stdio.h>
//#include<sys/type.h> #include<unistd.h>
void main()
{
{
int pid, retnice;
printf("press DEL to stop process \n");
pid=fork(); for(;;)
if(pid == 0)
{
retnice = nice (-5);
printf("child gets higher CPU priority %d \n", retnice); sleep(1);
}
```

```
else
{
retnice=nice(4);
printf("Parent gets lower CPU priority %d \n", retnice); sleep(1);
}
}
}
```

OUTPUT

press DEL to stop process Parent gets lower CPU priority 4 child gets higher

CPU priority -1 child gets higher CPU priority -1 Parent gets lower CPU priority

8 child gets higher CPU priority -1 Parent gets lower CPU priority 12 child gets

higher CPU priority -1 Parent gets lower CPU priority 16 child gets higher CPU

priority -1 Parent gets lower CPU priority 19

Parent gets lower CPU priority 19

Assignment No:1 Set: B (1)

Assignment Name: (1) Implement the C program to accept n integers to be sorted. Main function creates child process using fork system call. Parent process sorts the integers using bubble sort and waits for child process using wait system call. Child process sorts the integers using insertion sort.

```
# include<stdio.h>
# include <stdlib.h>
# include<sys/types.h>
#include<unistd.h>
void insert(int a[], int n);
void mergeSort(int arr[],int low,int mid,int high)
{
int i,j,k,l,b[20];
```

```
l=low; i=low;
j=mid+1;
while((<=mid)&&(j<=high)){
if(arr[l]<=arr[j]){
b[i]=arr[l];
l++;
} else{
b[i]=arr[j];
j++;
}
i++;
}
}
if(l>mid){
for(k=j;k<=high;k++){
else{
}
b[i]=arr[k];
i++;
for(k=l;k<=mid;k++){
b[i]=arr[k];
i++;
}
for(k=low;k<=high;k++)
{
arr[k]=b[k];
```

```
}
}
void partition(int arr[],int low, int high)
int mid;
if(low<high)
double temp;
mid=(low+high)/2;
partition(arr,low, mid);
partition(arr,mid+1,high);
mergeSort(arr,low,mid,high);
void display(int a[], int size){
int i;
for(i=0;i<size;i++){
}
printf("%d\t\t",a[i]);
printf("\n");
}
int main()
{
int pid, child_pid;
int size, i,status;
/* Input the Integers to be sorted */
printf("\nEnter the number of Integers to Sort::::\t");
scanf("%d",&size);
int a[size];
int pArr[size];
```

```
int cArr[size];
for(i=0;i<size;i++){
printf("\nEnter number %d:", (i+1));
scanf("%d",&a[i]);
pArr[i]=a[i];
cArr[i]=a[i];
}
/* Display the Enterd Integers */
printf("Your Entered Integers for Sorting\n");
display(a,size);
/* Process ID of the Parent */
pid=getpid();
printf("\nCurrent Process ID is: %d\n",pid);
/* Child Process Creation */
printf("\n[ Forking Child Process... ] \n");
child_pid=fork(); /* This will Create Child Process and
Returns Child's PID */
if( child_pid < 0){
/* Process Creation Failed ... */
printf("\nChild Process Creation Failed!!!!\n");
exit(-1);
else if( child_pid==0) {
printf("\nThe Child Process\n");
process is %d",getpid());
process is %d",getppid());
the list of Integers by::\n");
```

```
insert(cArr,size);
/* Child Process */
printf("\nchild
printf("\nparent of child
printf("\nChild is sorting
printf("\nThe sorted List by Child::\n");
display(cArr,size);
printf("\nChild Process Completed ...\n");
sleep(10);
printf("\nparent of child process is %d",getppid());
}
else {
/* Parent Process */ printf("\nparent
process %d started\n",getpid());
printf("\nParent of parent is %d\n",getppid());
sleep(30);
printf("\nThe Parent Process\n");
printf("\nParent %d is sorting the list of Integers by MERGE SORT\n", pid);
partition(pArr,0,size-1);
printf("\nThe sorted List by Parent::\n");
display(pArr,size);
wait(&status);
printf("\nParent Process Completed ...\n");
}
return 0;
}
```

```
void insert(int a[], int n) /* function to sort an aay with insertion sort */
int i, j, temp;
printf("\nUsing Insertion Sort");
for (i= 1; i <n; i++) {
temp = a[i];
j=i-1;
while(j>=0 && temp <= a[j]) /* Move the elements greater than temp to one position
ahead from their current position*/
a[j+1] = a[];
j = j-1;
}
a[j+1] = temp;
}
}
```

OUTPUT

```
Enter the number of Integers to Sort:::: 5
Enter number 1:5
Enter number 2:4
Enter number 3:8
Enter number 4:1
Enter number 5:9
Your Entered Integers for Sorting 5 4 8 1 9
Current Process ID is: 6429
[Forking Child Process ... ]
parent process 6429 started
Parent of parent is 5943
```

The Child Process

child process is 6430 parent of child

process is 6429 Child is sorting the

list of Integers by::

Using Insertion Sort The

sorted List by Child::

1 4 5 8 9

Child Process Completed ...

parent of child process is 6429

Assignment No:1 Set: B (2)

Assignment Name: Write a C program to illustrate the concept of orphan process. Parent process creates a child and terminates before child has finished its task. So child process becomes orphan process. (Use fork(), sleep(), getpid(), getppid()).

```
# include <stdio.h> #include<unistd.h>

#include<sys/types.h>

int main()

{ int pid;

pid=getpid();

printf("Current Process ID is: %d\n",pid);

printf("\n[Forking Child Process... ] \n");

pid=fork(); /* This will Create Child Process and Returns Child's PID */

if(pid < 0)

}

}

printf("\nProcess can not be created ");

//exit(-1);

else
```

```
if(pid==0)
{/* Child Process */
printf("\nChild Process is Sleeping ..."); sleep(5);
/*Orphan Child's Parent ID is 1 */ printf("\nOrphan
Child's Parent ID: %d",getppid());
}
else
{/* Parent Process */
printf("\nParent Process Completed ...");
}
}
}
return 0;
```

OUTPUT

```
Current Process ID is: 6681
[Forking Child Process...]
Parent Process Completed...
Child Process is Sleeping...
Orphan Child's Parent ID: 1674
```

Assignment No:2 Set: A (1)

Assignment Name: Write a C program that behaves like a shell which displays the command prompt 'myshell$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'count' as myshell$ count c filename: It will display the number of characters in given file myshell$ count w filename: It will display the number of words in given file myshell$ count l filename: It will display the number of lines in given file

```
#include<stdio.h>
```

```
#include<sys/types.h>
#include<sys/stat.h>
#include<unistd.h>
#include<dirent.h>
#include<fcntl.h>
#include<string.h> void
count(char c, char *fn)
{
int lc=0,wc=0,cc=0,handle;
char ch;
if((handle=open(fn,O_RDONLY))==-1)
printf("File %s not found\n",fn); return;
while(read(handle, &ch,1)!=0)
{
cc++;
/* Check new line */
if (ch == '\n' || ch == '\0')
lc++;
/* Check words */
if (ch =='' || ch == '\t' || ch == '\n' || ch == '\0')
Wc++;
close(handle);
switch(c)
{ case
'c':
printf("Total No.of Characters = %d\n",cc); break;
```

```
case 'w': printf("Total No.of Words =
%d\n",wc); break; case 'l':
}
{
printf("Total No.of Lines = %d\n",lc); break;
void main()
char command[80],t1[20],t2[20],t3[20],t4[20]; int
n;
//system("clear");
while(1)
printf("myShell$"); fflush(stdin);
fgets(command,80,stdin); n =
sscanf(command,"%s %s %s %s",t1,t2,t3,t4);
switch(n)
case 1:
if(!fork())
execlp(t1,t1, NULL);
perror(t1);
break;
case 2:
if(!fork())
{
execlp(t1,t1,t2,NULL);
}
perror(t1);
break;
```

```
case 3:
if(strcmp(t1,"count")==0
) count(t2[0],t3);
else
if(!fork())
execlp(t1,t1,t2,t3,NULL);
perror(t1);
}
}
break;
case 4:
if(!fork())
{
execlp(t1,t1,t2,t3,t4,NULL); perror(t1);
}
}
}
}
myShell$count c data.txt Total
No.of Characters = 118
myShell$count w data.txt
Total No.of Words = 14
myShell$count l data.txt
Total No.of Lines = 6
```

Assignment No:2 Set: B

Assignment Name: Write a C program that behaves like a shell which displays the command prompt 'myshell$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'list' as myshell$ list f dirname: It will display filenames in a given directory. myshell$ list n dirname: It will count the number of entries in a given directory. myshell$ list i dirname: It will display filenames and their inode number for the files in a givendirectory.

```
#include<stdio.h>

#include<sys/types.h>

#include<sys/stat.h>

#include<unistd.h>

#include<dirent.h>

#include<fcntl.h>

#include<string.h>

void list(char c, char *dn)

{

}

DIR *dir; int cnt=0;

struct dirent *entry;

struct stat buff;

if((dir=opendir(dn))==NULL)

printf("Directory %s not found\n",dn); return;

}

switch(c)

{case

{

}

'f':
```

```
while((entry=readdir(dir))!=NULL)
stat(entry->d_name,&buff);
if(S_IFREG&buff.st_mode)
printf("%s\n",entry->d_name);
break;
case 'n':
while((entry=readdir(dir))!=NULL) cnt++;
printf("Total No.of Entries = %d\n",cnt);
break;
case 'i':
while((entry=readdir(dir))!=NULL)
stat(entry->d_name,&buff);
if(S_IFREG&buff.st_mode)
printf("%s\t%ld\n",entry->d_name,buff.st_ino);
}
break; default: printf("Invalid
argument...\n");
}
closedir(dir);
}
void main()
char command[80],t1[20],t2[20],t3[20],t4[20]; int
n;
//system("clear");
while(1)
{
```

```
printf("myShell$"); fflush(stdin);
fgets(command,80,stdin); n =
sscanf(command,"%s %s %s %s",t1,t2,t3,t4);
switch(n)
case 1:
if(!fork())
execlp(t1,t1,NULL);
perror(t1);
break;
case 2:
if(!fork())
{
execlp(t1,t1,t2,NULL);
}
perror(t1);
break;
case 3:
if(strcmp(t1,"list")==0)
list(t2[0],t3); else
if(!fork())
{
execlp(t1,t1,t2,t3,NULL);
perror(t1);
}
}
break;
```

```
case 4:
if(!fork())
{
}
execlp(t1,t1,t2,t3,t4,NULL);
perror(t1);
}
}
```

OUTPUT

```
myShell$list f Assignment-2
psjf.c
npsjf1.c
preemitivepriority.c
* nonpreemitivepriority.c
rrm1.c
* fcfs1.c
* myShell$list n Assignment-2
Total No.of Entries = 8
myShell$list i Assignment-2
psjf.c 546949
npsjf1.c 546955
preemitivepriority.c 547134
nonpreemitivepriority.c 547134
* rrm1.C 546960
* fcfs1.c 546904
myShell$
```

Assignment No:2 Set: C (1)

Assignment Name: Write a C program that behaves like a shell which displays the command prompt 'myshellŞ'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'typeline' as myshell$ typeline n filename: It will display first n lines of the file. myshell$ typeline -n filename: It will display last n lines of the file. myshell$ typeline a filename: It will display all the lines of the file.

```
#include<stdio.h>

#include<sys/types.h>

#include<sys/stat.h>

#include<unistd.h>

#include<dirent.h>

#include<fcntl.h>

#include<string.h>

#include<stdlib.h>

void

}

 typeline(char *s, char *fn)

int handle,i=0,cnt=0,n; char

ch;

if((handle=open(fn,O_RDONLY))==-1)

printf("File %s not found\n",fn); return;

}

{

if(strcmp(s,"a")==0)

while(read(handle, &ch,1)!=0)

printf("%c",ch); close(handle);
```

```
return;
}
}
n=atoi(s);
if(n>0)
while(read(handle,&ch,1)!=0)
}
if(ch=='\n')
i++; if(i==n)
break;
printf("%c",ch);
}
printf("\n"); close(handle);
}
{
return;
if(n<0)
while(read(handle,&ch,1)!=0)
{
if(ch=='\n') cnt++;
}
lseek(handle,O,SEEK_SET);
while(read(handle, &ch,1)!=0)
{
if(ch=='\n') i++;
if(i==cnt+n)
```

```
break;
}
while(read(handle, &ch,1)!=0
) printf("%c",ch); printf("\n");
close(handle);
}
}
void main()
char command[80],t1[20],t2[20],t3[20],t4[20]; int
n;
//system("clear");
while(1)
{
printf("myShell$");
fflush(stdin); fgets(command,80,stdin); n =
sscanf(command,"%s %s %s %s",t1,t2,t3,t4);
switch(n)
case 1:
if(!fork())
execlp(t1,t1, NULL);
perror(t1);
}
break;
case 2:
if(!fork())
{
```

```
execlp(t1,t1,t2,NULL); perror(t1);
}
break; case 3:
if(strcmp(t1, "typeline")==0)
{
typeline(t2,t3); else
if(!fork())
{
execlp(t1,t1,t2,t3,NULL); perror(t1);
}
}
break;
case 4:
if(!fork())
{
execlp(t1,t1,t2,t3,t4,NULL); perror(t1);
}
}
}
}
```

OUTPUT

```
myShell$list f Assignment1 seta1.c setb1.c a.out seta2.c
myShell$list n Assignment1
Total No.of Entries = 7
myShell$list i Assignment1
seta1.c 140731229406416
```

setb1.c 140731229406416

a.out 3024575 seta2.c

3024575

Assignment No:2 Set: C (2)

Assignment Name: Write a C program that behaves like a shell which displays the command prompt 'myshell$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'search' as myshell$ search f filename pattern: It will search the first occurrence of pattern in the given file myshell$ search a filename pattern: It will search all the occurrence of pattern in the given file myshell$ search c filename pattern: It will count the number of occurrence of pattern in the given file

```
#include<stdio.h>

#include<sys/types.h>

#include<sys/stat.h>

#include<unistd.h>

#include<dirent.h>

#include<fcntl.h>

#include<string.h>

void search(char c, char *s, char *fn)

{

int handle,i=1,cnt=0,j=0; char

ch,buff[80],*p;

if((handle=open(fn,O_RDONLY))==-1)

{

printf("File %s not found\n",fn); return;

{

switch(c)

{case
```

```
'f':
while(read(handle, &ch,1)!=0)
if(ch=='\n')
buff[j]='\0'; j=0;
if(strstr(buff,s)!=NULL)
printf("%d : %s\n",i,buff); break;
i++;
}
}
else buff[j++]=ch;
break;
case 'c':
while(read(handle,&ch,1)!=0)
{
{
}
if(ch=='\n')
buff[j]='\0'; j=0;
if(strstr(buff,s)!=NULL)
p=buff;
while((p=strstr(p,s))!=NULL)
cnt++; p++;
}
}
}
else buff[j++]=ch;
```

```
printf("Total No.of Occurrences = %d\n",cnt);
break; case 'a':
while(read(handle, &ch,1)!=0)
if(ch=='\n')
buff[j]='\0'; j=0;
if(strstr(buff,s)!=NULL)
printf("%d : %s\n",i,buff);
i++;
}
else
buff[j++]=ch;
}
}
close(handle);
}
void main()
char command[80],t1[20],t2[20],t3[20],t4[20]; int
n;
//system("clear");
while(1)
{
printf("myShell$"); fflush(stdin);
fgets(command,80,stdin); n =
sscanf(command,"%s %s %s %s",t1,t2,t3,t4);
switch(n)
{
```

```
{
}
case 1:
if(!fork())
execlp(t1,t1, NULL);
perror(t1);
break;
case 2:
if(!fork())
{
execlp(t1,t1,t2,NULL);
}
{
perror(t1);
break;
case 3:
if(!fork())
execlp(t1,t1,t2,t3,NULL);
perror(t1);
}
{
break;
case 4:
if(strcmp(t1,"search")==0) search(t2[0],t3,t4);
else
if(!fork())
```

```
{
execlp(t1,t1,t2,t3,t4,NULL); perror(t1);
}
}
}
}
*****Input:data.tx+***
System Programming
Programming in Java
Internet Programming
```

OUTPUT

myShell$search f Pro data.txt 1: System Programming myShell$search a Pro data.txt

1: System Programming

2: Programming in Java 3: Internet Programming

myShell$search c Pro data.txt

Total No.of Occurrences = 3

Assignment No:3 Set: A (1)

Assignment Name: Write the program to simulate FCFS CPU-scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. Assume that the fixed IO waiting time (2 units). The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```
#include<stdio.h>
void main()
{
int n,i,a[20],b[20],c[20],f[20],w[20],t[20],s[20],m=0; float
tat=0.0,wt=0.0;
```

```
char ch; printf("\n Enter the no
ofjobs");
scanf("%d",&n);
printf("\n Is there Arrival Time(y/n)?");
//fflush(stdin);
}
ch=getchar();
scanf("%c",&ch);
printf("\n%c",ch); if(ch=='y')
for(i=0;i<n;i++)
printf("Enter the arrival time of process %d:",i); scanf("%d",&a[i]);
printf("\nEnter the Starting time"); for(i=0;i<n;i++)
printf("\nEnter the starting time of process %d:",i); scanf("%d",&s[i]);
}
printf("\nEnter the Burst Time:");
for(i=0;i<n;i++)
{
}
printf("\nEnter the Burst time of process %d:",i);
scanf("%d",&b[i]);
printf("\n JOB | CPU BURST TIME | ARRIVAL TIME\n");
printf(" --\n");
for(i=0;i<n;i++)
}
{
if(ch=='y')
```

```
printf("\n%5d %10d %10d",i+1,b[i],a[i]);
}
}
printf("\n-
}
else
--\n");
{
{
printf("\nEnter the Starting time"); for(i=0;i<n;i++);
printf("\nEnter the starting time of process %d:",i); scanf("%d",&s[i]);
}
{
}
printf("\nEnter the Burst Time:");
for(i=0;i<n;i++)
printf("\nEnter the Burst time of process %d:",i);
 scanf("%d",&b[i]);
printf("\n JOB | CPU BURST TIME | ARRIVAL TIME\n");

printf("---\n");
for(i=0;i<n;i++)
{
/*if(ch=='y')
* {
* printf("\n%5d %10d %10d",i+1,b[i],a[i]);
```

```
*}*/
printf("\n %5d %10d %10d",i+1,b[i],0);
}
}
printf("\nfor(i=0;i<n;i++)
{
{
if(ch=='y')
w[i]=s[i]-a[i);
c[i]=s[i]+b[i]; t[i]=c[i]-
a[i]; wt=wt+w[i];
tat=tat+t[i];
}
else
--\n");
w[i]=m; c[i]=m=m+b[i];
t[i]=c[i];
wt=wt+w[i]; tat=tat+t[i];
}
printf("\n\n\nJob | Turn around time | Waiting time\n");
printf("
for(i=0;i<n;i++)
{
--\n");
printf("\n%5d%10d%10d",i+1,t[i],w[i]);
printf("\n----------------");
```

printf("\n\n\nAverage turn around time=%f",tat/n); printf("\n\n\nAverage

waiting time=%f",wt/n);

OUTPUT

Enter the no of jobs5

Is there Arrival Time(y/n)?y

yEnter the arrival time of process 0:1

Enter the arrival time of process 1:0

Enter the arrival time of process 2:2

Enter the arrival time of process 3:3

Enter the arrival time of process 4:2

Enter the Starting time

Enter the starting time of process 0:8

Enter the starting time of process 1:3

Enter the starting time of process 2:10

Enter the starting time of process 3:22

Enter the starting time of process 4:18

Enter the Burst Time:

Enter the Burst time of process 0:5

Enter the Burst time of process 1:3

Enter the Burst time of process 2:2

Enter the Burst time of process 3:4

Enter the Burst time of process 4:8

JOB | CPU BURST TIME | ARRIVAL TIME

1 5 1

2 3 0

3 2 2

4 4 3

5 8 2

Job | Turn around time | Waiting time

1 12 7

2 6 3

3 10 8

4 23 19

5 24 16

Average turn around time=15.000000

Average waiting time=10.600000

Assignment No:3 Set: A (2)

Assignment Name: Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. Assume the fixed IO waiting time (2 units). The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```
#include<stdio.h>

void main()

int i,n,p[10]={1,2,3,4,5,6,7,8,9,10},min,k=1,btime=0; int

bt[10],temp,j,at[10],wt[10],tt[10],ta=0,sum=0,st[10];

float wavg=0,tavg=0,tsum=0,wsum=0;

printf("-------Shortest Job First Scheduling (NP )-------\n"); printf("\nEnter the No. of processes :");

scanf("%d",&n);

for(i=0;i<n;i++)

printf("\tEnter the burst time of %d process :",i+1);

scanf("%d",&bt[i]);
```

```
}
printf("\tEnter the arrival time of %d process :",i+1);
 scanf("%d",&at[i]);
/Sorting According to Arrival Time/
for(i=0;i<n;i++)
{
for(j=0;j<n;j++)
***********/
{
if(at[i]<at[j])
{
temp=p[j];
p[j]=p[i]; p[i]=temp;
temp=at[j];
at[j]=at[i];
at[i]=temp;
temp=bt[j):
bt[j]=bt[i];
bt[i]=temp;
}
if(at[i]==at[j])
{if(bt[i]<bt[j])
{
temp=p[j];
p[j]=p[i]; p[i]=temp;
temp=at[j);
```

```
at[j]=at[i];
at[i]=temp;
temp=bt[j;
bt[j]=bt[i];
bt[i]=temp;
if(bt[i]==bt[j])
{ p[i];
pil;
at[i];
at[j];
bt[i];
bt[j];
}
}
}
}
/*Arranging the table according to Burst time,
* Execution time and Arrival Time
* Arrival time <= Execution time
**/
for(j=0;j<n;j++)
{
{
btime=btime+bt[j];
min=bt[k]; for(i=k;i<n;i++)
if (btime>=at[i] && bt[i]<min)
```

```
{
temp=p[k];
p[k]=p[i]; p[i]=temp;
temp=at[k];
at[k]=at[i];
at[i]=temp;
temp=bt[k];
bt[k]=bt[i;
bt[i]=temp;
}
}
}
k++;
wt[0]=0;
for(i=1;i<n;i++)
sum=sum+bt[i-1];
wt[i]=sum-at[i];
wsum=wsum+wt[i];
}
wavg=(wsum/n);
for(i=0;i<n;i++)
{
ta=ta+bt[i]; st[i]=ta;
tt[i]=ta-at[i];
tsum=tsum+tt[i];
}
```

```
tavg=(tsum/n);
printf("********"); printf("\n
RESULT:-");
{
}
printf("\nProcess\t Burst\t Arrival\t Waiting\t Turn-around \t Finish Time"); for(i=0;i<n;i++)
printf("\n p%d\t %d\t %d\t\t %d\t\t\t%d\t\t\t%d",p[i],bt[i],at[i],wt[i],tt[i],st[i]);
printf("\n\nGANTT CHART");
printf("\n
for(i=0;i<n;i++) printf(" \tp%d\t",p[i]);
printf("\t|");
printf("\n_
for(i=0;i<n;i++)
printf("\t%d\t",st[i]);
}
\n");
\n"); printf("%d\t",wt[O]);
printf("\n\nAVERAGE WAITING TIME: %f", wavg);
printf("\nAVERAGE TURN AROUND TIME: %f",tavg);
}
```

OUTPUT

```
-------Shortest Job First Scheduling (NP)-------
Enter the No. of processes :4
Enter the burst time of 1 process :4
Enter the arrival time of 1 process :0
Enter the burst time of 2 process :1
```

Enter the arrival time of 2 process :1

Enter the burst time of 3 process :2

Enter the arrival time of 3 process :2

Enter the burst time of 4 process :1

Enter the arrival time of 4 process :3

RESULT:-

Process Burst Arrival Waiting Turn-around Finish Time

p1 4 0| 0| 4 4

p2 1 1 3 4 5

p4 1 3 2 3 6

p3 2 2 4 6 8

GANTT CHART

p1 p2 p4 p3

0 4 5 6 8

AVERAGE WAITING TIME: 2.250000

AVERAGE TURN AROUND TIME: 4.250000

Assignment No:3 Set: B (1)

Assignment Name: Write the program to simulate Preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. Assume the fixed IO waiting time (2 units). The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```
#include<stdio.h>

#include<string.h> #include<stdlib.h>

struct Input

{

char pname[10];
```

```
int bt,at,ct,tbt;
}tab[5];
struct Sequence
{
int start,end; char
pname[10];
}seq[100],seq1[20];
int finish,time, n,k,prev;
void getinput()
{
int i;
{
printf("\nEnter No.of Processes:"); scanf("%d",&n);
for(i=0;i<n;i++)
printf("Process name:");
scanf("%s",tab[i].pname);
printf("Burst time:");
scanf("%d",&tab[i].bt); printf("Arrival
time:"); scanf("%d",&tab[i].at);
tab[i].tbt = tab[i].bt;
}
}
void printinput()
{
int i;
printf("\nThe processes Snapshot\n");
```

```
printf("\nprintf("\nProcess Burst Time-");
Arrival Time"); printf("\n----"); for(i=0;i<n;i++)
printf("\n%5s\t%20d\t%15d",tab[i].pname,tab[i].tbt,tab[i].at);
printf("\n- -\n");
}
int arrived()
{
int i;
for(i=0;i<n;i++) if(tab[i].at<=time
&& tab[i].tbt!=0) return 1; return
0;
}
{
{
}
}
int getmin(int t)
int i, mini, min=99; for(i=0;i<n;i++)
if(tab[i].at<=t && tab[i].tbt!=0 && tab[i].tbt<min)
min = tab[i].tbt;
mini = i;
return mini;
void sort()
{
struct Input t;
int i,j;
```

```
for(i=0;i<n;i++) for(j=0;j<n-i-1;j++)
if(tab[j].at > tab[j+1].at)
{t = tab[j];
tab[j] = tab[j+1];
tab[j+1] = t;
}
}
void processinput()
{
{
int i;
finish=k=0; while(finish!=n)
printf("\n\nTime=%d",time);
if(arrived(time))
{
i = getmin(time);
time++;
tab[i].tbt--; tab[i].ct=time;
seq[k].start=prev; seq[k].end = time;
strcpy(seq[k++].pname,tab[i].pname);
prev = time;
if(tab[i].tbt==0)
{
finish++;
}
}
```

```
else
{
time++; seq[k].start=prev;
seq[k].end = time;
strcpy(seq[k++].pname,"*");
prev = time;
}
}
}
{
void printoutput()
int i;
float AvgTAT=0,AvgWT=0; printf("\nProcess\tAT\tBT\tCT\tTAT\tWT");
for(i=0;i<n;i++)
{
printf("\n%s\t%d\t%d\t%d\t%d\t%d",tab[i].pname,
tab[i].at, tab[i].bt, tab[i].ct,
tab[i].ct-tab[i].at, tab[i].cttab[i].at-tab[i].bt); AvgTAT
+= tab[i].ct-tab[i].at;
AvgWT += tab[i].ct-tab[i].at-tab[i].bt;
}
AvgTAT/=n; AvgWT/=n;
printf("\nAverage TAT = %f",AvgTAT);
printf("\nAverage WT = %f",AvgWT);
}
void ganttchart()
```

```
{
{
int i,j=1; seq1[0]
= seq[0];
for(i=1;i<k;i++)
if(strcmp(seq1[j-1].pname,seq[i].pname)==0)
seq1[j-1].end = seq[i].end;
else seq1[j++] = seq[i];
}
}
printf("\n\nGantt Chart\n");
printf("\nprintf("| %s ",seq1[i].pname);
printf("|\nprintf("%d\t",seq1[i].start);
-\n"); for(i=0;i<j;i++)
-\n"); for(i=0;i<j;i++)
}
printf("%d\t",seq1[j-1].end);
printf("\n-- --"); printf("\n");
}
void main()
{int
i;
getinput();
printinput(); sort();
processinput();
printoutput();
```

```
ganttchart();
for(i=0;i<n;i++)
tab[i].tbt = tab[i].bt=rand()%10+1;
tab[i].at=tab[i].ct+2;
}}
Enter No.of Processes:4
Process name:p1
Burst time:8
Arrival time:0
Process name:p2
Burst time:4
Arrival time:1
Process name:p3
Burst time:9
Arrival time:2
Process name:p4
Burst time:5
Arrival time:3
The processes Snapshot
Process Burst Time Arrival
Time
---- p1 8 0 p2
4 1
p4
p3
5
```

9 2

3

Time=0

Time=1

Time=2

Time=3

Time=4

Time=5

Time=6

Time=7

Time=8

Time=9

Time=10

Time=11

Time=12

Time=13

Time=14

Time=15

Time=16

Time=17

Time=18

Time=19

Time=20

Time=21

Time=22

Time=23

Time=24

Time=25

Process AT BT CT TAT WT

p1 0 8 17 17 9

p2 1 4 5 4 0|

p3 2 9 26 24 15

p4 3 5 10 7 2

Average TAT = 13.000000

Average WT = 6.500000

Gantt Chart

p1 p2 | p4 | p1 | p3

0 1 5 10 17 26

Assignment No:3 Set: B (2)

Assignment Name: Write the program to simulate Non-preemptive Priority scheduling. The arrival time and first CPU-burst and priority for different n number of processes should be input to the algorithm. Assume the fixed IO waiting time (2 units). The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```
#include<stdio.h>

#include<string.h>

#include<stdlib.h>

struct Input

char pname [10];

int bt,at,ct,p,tbt;

}tab[5];

struct Sequence

int start,end; char
```

```
pname[10];
}seq[100],seq1[20];
int finish,time,n,k,prev;
void getinput()
{int
i;
printf("\nEnter No.of Processes:"); scanf("%d",&n);
for(i=0;i<n;i++)
{
printf("Process name:");
scanf("%s",tab[i].pname);
printf("Burst time:");
scanf("%d",&tab[i].bt); printf("Arrival
time:"); scanf("%d",&tab[i].at);
printf("Priority level:");
scanf("%d",&tab[i].p);
tab[i].tbt = tab[i].bt;
}
}
void printinput()
{
int i;
printf("\nProcess\tBT\tAT\tPriority");
for(i=0;i<n;i++)
printf("\n%s\t%d\t%d\t%d",tab[i].pname,tab[i].tbt,tab[i].at,tab[i].p);
}
```

```
void sort()
{
struct Input t;
int i,j;
for(i=0;i<n;i++) for(j=0;j<
n-1-i;j++)
if(tab[j+1].p<tab[j].p)
{t = tab[i];
tab[i] =
tab[j];
tab[j] = t;
}
void processinput()
int i;
k=finish=0;
while(finish!=n)
{
outside:
for(i=0;i<n;i++)
{
{
if(tab[i].at<=time && tab[i].tbt!=0)
while(tab[i].tbt!=0)
{
time++;
tab[i].tbt--;
```

```
printinput();
}
seq[k].start=prev; seq[k].end
= time;
strcpy(seq[k++].pname,tab[i].pname);
prev = time;
tab[i].ct=time;
finish++; goto
outside;
}}
{
if(finish!=n)
time++; seq[k].start=prev;
seq[k].end = time;
strcpy(seq[k++].pname,"*");
prev = time;
}
}
}
void printoutput()
{
int i;
float AvgTAT=0,AvgWT=0; printf("\nProcess\tAT\tBT\tCT\tTAT\tWT");
for(i=0;i<n;i++)
printf("\n%s\t%d\t%d\t%d\t%d\t%d",tab[i].pname,
tab[i].at, tab[i].bt, tab[i].ct, tab[i].ct-tab[i].at,
```

```
tab[i].ct-tab[i].at-tab[i].bt); AvgTAT += tab[i].cttab[i].at;
AvgWT += tab[i].ct-tab[i].at-tab[i].bt;
}
AvgTAT/=n; AvgWT/=n;
}
}
printf("\nAverage TAT = %f",AvgTAT);
printf("\nAverage WT = %f",AvgWT);
void ganttchart()
int i,j=1; seq1[0]
= seq[0];
for(i=1;i<k;i++)
{
if(strcmp(seq1[j-1].pname,seq[i].pname)==0)
seq1[j-1].end = seq[i].end; else
seq1[j++] = seq[i];
}
for(i=0;i<j;i++)
printf("\n%d\t%s\t%d",seq1[i].start,seq1[i].pname,seq1[i].end);
}
void main()
{int
i;
{
getinput(); sort();
processinput();
```

```
printoutput();
ganttchart();
for(i=0;i<n;i++)
tab[i].tbt = tab[i].bt=rand()%10+1;
tab[i].at=tab[i].ct+2;
}
}
```

OUTPUT

```
Enter No.of Processes:4
Process name:p1
Burst time:2
Arrival time:0
Priority level:3
Process name:p2
Burst time:3
Arrival time:1
Priority level:1
Process name:p3
Burst time:4
Arrival time:2
Priority level:4
Process name:p4
Burst time:3
Arrival time:3
Priority level:2
Process BT AT Priority
```

p2 3 1 1

p3 4 2 4

p1 1 0 3

p4 3 3 2

Process BT AT Priority

p2 3 1 1

p3 4 2 4

p1 0| 0| 3

p4 3 3 2

Process BT AT Priority

p2 2 1 1

p3 4 2 4

p1 0 0 3

p4 3 3 2

Process BT AT Priority

p2 1 1 1

p3 4 2 4

p1 0 0 3

p4 3 3 2

Process BT AT Priority

p2 0| 1 1

p3 4 2 4

p1 0 0| 3

p4 3 3 2

Process BT AT Priority

p2 0| 1 1

p3 3 2 4

p1 0 0 3

p4 3 3 2

Process BT AT Priority

p2 0| 1 1

p3 2 2 4

p1 0| 0 3

p4 3 3 2

Process BT AT Priority

p2 0 1 1

p3 1 2 4

p1 0| 0 3

p4 3 3 2

Process BT AT Priority

p2 0 1 1

p3 0 2 4

p1 0 0| 3

p4 3 3 2

Process BT AT Priority

p2 0 1 1

p3 0 2 4

p1 0| 0 3

p4 2 3 2

Process BT AT Priority

p2 0 1 1

p3 0| 2 4

p1 0| 0| 3

p4 1 3 2

Process BT AT Priority

p2 0| 1 1

p3 0| 2 4

p1 0| 0| 3

p4 0 3 2

Process AT BT CT TAT WT

p2 1 3 5 4 1

p3 2 4 9 7 3

p1 0 2 2 2 0

p4 3 3 12 9 6

Average TAT = 5.500000

Average WT = 2.500000

0| p1 2

2 p2 5

5 p3 9

9 p4 12

Assignment No:3 Set: C (1)

Assignment Name: Write the program to simulate Preemptive Priority scheduling. The arrival time and first CPU-burst and priority for different n number of processes should be input to the algorithm. Assume the fixed IO waiting time (2 units). The next CPU-burst

should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
 Char processname[5][10];
Int bursttime[5],arrivaltime[5],ttime[5],priority[5],tempbursttime[5];
int
finish, length, time,k,prev;
{
struct Sequence
int start,end; char
processname[10];
}seq[100], seq1[20];
void getinput()
{
int x;
printf("Enter No.of Proesses:"); scanf("%d",&length);
for(x=0;x<length;x++)
{
printf("Process name:");
scanf("%s",processname[x]);
printf("Burst time:");
scanf("%d",&bursttime[x]); printf("Arrival
time:"); scanf("%d",&arrivaltime[x]);
printf("Priority level:");
```

```
scanf("%d",&priority[x]);
}
}
void coutinput()
{
int x;
printf("\nProcess\tBT\tAT\tPriority"); for(x=0;x<length;x++)
printf("\n%s\t%d\t%d\t%d",processname[x],bursttime[x],arrivaltime[x],priority[x]);
}
void tcoutinput()
int x;
{
printf("\nProcess\tBT\tAT\tPriority"); for(x=0;x<length;x++)
printf("\n%s\t%d\t%d\t%d",processname[x],tempbursttime[x],arrivaltime[x],priority[x]);
}
{
void swap(int x,int y)
int temp;
char temp1[10];
temp=bursttime[x];
bursttime[x]=bursttime[y];
bursttime[y]=temp;
temp=priority[x]; priority[x]=priority[y];
priority[y]=temp;
strcpy(temp1,processname[x]); strcpy(processname[x], processname[y]);
strcpy(processname[y],temp1);
```

```
temp=arrivaltime[x]; arrivaltime[x]=arrivaltime[y];
arrivaltime[y]=temp;
}
void bubble()
int i,j;
for(i=0;i<length;i++)
{
for(j=0;j<(length-1-i);j++)
if(priority[j+1]<priority[j])
{
swap(j+1,j);
}
}
}
}
void copy()
int x;
for(x=0;x<length;x++)
{
tempbursttime[x]=bursttime[x];
}
}
void outputprocess()
{
int x;
float AvgTAT=0,AvgWT=0;
```

```
}
printf("\nProcess\tTAT\tWT"); for(x=0;x<length;x++)
printf("\n%s\t%d\t%d",processname[x], ttime[x]-
arrivaltime[x], ttime[x]-arrivaltime[x]-bursttime[x]);
AvgTAT+=ttime[x]-arrivaltime[x];
AvgWT+=ttime[x]-arrivaltime[x]-bursttime[x];
}
printf("\nAverage TAT = %f", AvgTAT/length);
printf("\nAverage WT = %f",AvgWT/length);
}
;
void processinput()
{ int x; finish=0; k=0;
printf("\n\ntime=%d",time)
while(finish!=length)
{
outside:
for(x=0;x<length;x++)
{
if(arrivaltime[x]<=time && tempbursttime[x]!=0)
time++;
tempbursttime[x]--;
ttime[x]=time;
tcoutinput();
seq[k].start=prev; seq[k].end
= time;
```

```
strcpy(seq[k++].processname,processname[x]);
prev = time;
if(tempbursttime[x]==0)
finish++;
goto outside;
}
}
if(finish!=length)
time++; seq[k].start=prev;
seq[k].end = time;
strcpy(seq[k++].processname,"*");
prev = time;
}
}
coutinput();
}
void ganttchart()
int x,l=1; seq1[0].start =
seq[0].start; seq1[0].end =
seq[0].end;
strcpy(seq1[0].processname,seq[0].processname);
for(x=1;x<k;x++)
if(strcmp(seq1[l-1].processname, seq[x].processname)==0)
seq1[l-1].end = seq[x].end;
else
seq1[l].start = seq[x].start; seq1[l].end
```

```
= seq[x].end;
strcpy(seq1[l++].processname,seq[x].processname);
for(x=0;x<l;x++)
}
printf("\n%d\t%s\t%d",seq1[x].start,seq1[x].processname,seq1[x].end);
}
void main()
{ int x;
getinput();
bubble();
coutinput();
copy();
processinput();
outputprocess();
ganttchart();
for(x=0;x<length;x++)
{
bursttime[x]=rand()%10+1;
arrivaltime[x]=ttime[x]+2;
}
}
Enter No.of Proesses:4
Process name:p1
Burst time:8
Arrival time:0
Priority level:4
```

Process name:p2

Burst time:6

Arrival time:1

Priority level:6

Process name:p3

Burst time:7

Arrival time:3

Priority level:3

Process name:p4

Burst time:9

Arrival time:3

Priority level:1

Process BT AT Priority

p4 9 3 1

p3 7 3 3

p1 8 0| 4

p2 6 1 6

time=0

ProcesS BT AT Priority

p4 9 3 1

p3 7 3 3

p1 7 0| 4

p2 6 1 6

Process BT AT Priority

p4 9 3 1

p3 7 3 3

p1 6 0| 4

p2 6 1 6

Process BT AT Priority

p4 9 3 1

p3 7 3 3

p1 5 0 4

p2 6 1 6

Process BT AT Priority

p4 8 3 1

p3 7 3 3

p1 5 0 4

p2 6 1 6

Process BT AT Priority

p4 7 3 1

p3 7 3 3

p1 5 0 4

p2 6 1 6

Process BT AT Priority

p4 6 3 1

p3 7 3 3

p1 5 0 4

p2 6 1 6

Process BT AT Priority

p4 5 3 1

p3 7 3 3

p1 5 0| 4

p2 6 1 6

Process BT AT Priority

p4 4 3 1

p3 7 3 3

p1 5 0 4

p2 6 1 6

Process BT AT Priority

p4 3 3 1

p3 7 3 3

p1 5 0| 4

p2 6 1 6

Process BT AT Priority

p4 2 3 1

p3 7 3 3

p1 5 0 4

p2 6 1 6

Process BT AT Priority

p4 1 3 1

p3 7 3 3

p1 5 0 4

p2 6 1 6

Process BT AT Priority

p4 0 3 1

p3 7 3 3

p1 5 0 4

p2 6 1 6

Process BT AT Priority

p4 0 3 1

p3 6 3 3

p1 5 0 4

p2 6 1 6

Process BT AT Priority

p4 0| 3 1

p3 5 3 3

p1 5 0 4

p2 6 1 6

Process BT AT Priority

p4 0 3 1

p3 4 3 3

p1 5 0| 4

p2 6 1 6

Process BT AT Priority

p4 0 3 1

p3 3 3 3

p1 5 0| 4

p2 6 1 6

Process BT AT Priority

p4 0| 3 1

p3 2 3 3

p1 5 0| 4

p2 6 1 6

Process BT AT Priority

p4 0| 3 1

p3 1 3 3

p1 5 0 4

p2 6 1 6

Process BT AT Priority

p4 0| 3 1

p3 0| 3 3

p1 5 0| 4

p2 6 1 6

Process BT AT Priority

p4 0| 3 1

p3 0 3 3

p1 4 0| 4

p2 6 1 6

Process BT AT Priority

p4 0 3 1

p3 0 3 3

p1 3 0| 4

p2 6 1 6

Process BT AT Priority

p4 0| 3 1

p3 0| 3 3

p1 2 0 4

p2 6 1 6

Process BT AT Priority

p4 0| 3 1

p3 0 3 3

p1 1 0 4

p2 6 1 6

Process BT AT Priority

p4 0| 3 1

p3 0| 3 3

p1 0| 0 4

p2 6 1 6

Process BT AT Priority

p4 0 3 1

p3 0 3 3

p1 0 0| 4

p2 5 1 6

Process BT AT Priority

p4 0| 3 1

p3 0 3 3

p1 0| 0| 4

p2 4 1 6

Process BT AT Priority

p4 0 3 1

p3 0| 3 3

p1 0 0| 4

p2 3 1 6

Process BT AT Priority

p4 0 3 1

p3 0 3 3

p1 0| 0 4

p2 2 1 6

Process BT AT Priority

p4 0 3 1

p3 0 3 3

p1 0 0| 4

p2 1 1 6

Process BT AT Priority

p4 0 3 1

p3 0| 3 3

p1 0| 0 4

p2 0 1 6

Process BT AT Priority

p4 9 3 1

p3 7 3 3

p1 8 0| 4

p2 6 1 6

Process TAT WT

p4 9 0

p3 16 9

p1 24 16

p2 29 23

Average TAT = 19.500000

Average WT = 12.000000

0 p1 3

3 p4 12

12 p3 19

19 p1 24

24 p2 30

Assignment No:3 Set: C (2)

Assignment Name: Write the program to simulate Round Robin (RR) scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. Also give the time quantum as input. Assume the fixed IO waiting time (2 units). The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround turnarm

//The index of each array denotes the (Process index - 1) except pq

int at[10], bt[10],rt[10],pq[10],pflag[10],tat[10],wt[10];

//Initially filling all the values as default

for(j=0;j<n;j++)

= 0;

{]j[qp

```
***/
pflag[j] = O;
}
//Scanning other data from the files
printf("The scanned process data is:\n\n");
printf("
printf("|\tINDEX\t|\tAT\t/\tBT\t /\n");
for(count=0;count<n;count++)
{
--\n");
fscanf(fp,"%d\t%d",&at[count],&bt[count]);
printf("|\t%d\t/\t%d\t/\t%d\t/\n",count+1,at[count],bt[count]);
rt[count]=bt[count]; t_check += bt[count]; //Check for invalid
data read if(at[count] < 0 || bt[count] <= 0)
{
printf("Invalid Data!");
return 0;
}
}
printf(" ------\n");
//The index of the gantt variable would represent the time instance
//The data number at the index would denote the process that was executed at that time
int gantt[200];
//Initially filled with default
values for(i=0;i<200;i++)
gantt[i] = 0;
```

```
//current variable is used to keep track of the time from which a new process
started execution int current = 0;
printf("\nThe order of execution:\n\n");
//Denotes the number of active process in the process queue
int running_processes = 0;
//Putting process 1 in the process queue for a
start pq[0] = 1; running_processes = 1;
pflag[0] = 1;
int flag = 0;
//Main loop
while(running_processes!=0)
{
flag = 0; //To represent weathera process has ended execution so it can be removed
from the process queue
//Printing status of pq and rt so we can see the changes taking
place printf("\nProcess queue: ");
for(i=0;i<running_processes;i++)
printf("%d ",pq[i]);
printf("\t");
printf("\nRemaining Time: ");
for(i=0;i<n;i++)
printf("%d ",rt[i]);
printf("\nRunning processes: %d\n",running_processes); //Running
the first process in queue(at index 0) in the process queue //If the
remaining time of the process is more than the burst time
if(rt[pq[0]-1]>time_quantum)
```

```
}
rt[pq[0]-1] = rt[pq[0]-1] - time_quantum;
time = time + time_quantum;
printf("Process %d executed for %d bursts till time %d\n",pq[0],time_quantum,time);
fill_gantt(gantt,current,time,pq[0]);
//current = time;
printf("Process=%d\tfrom=%d\tTo=%d\n",pq[0],current, time); current=time;
else
}
//if the remaining time is less than or equal to the burst time
//The process will finish it's execution and terminate
{
time = time + rt[pq[0]-1];
printf("Process %d executed for %d bursts till time %d\n",pq[0],rt[pq[0]-1],time);
rt[pq[0]-1] = 0; flag = 1;
fill_gantt(gantt,current,time,pq[0]);
printf("Process=%d\tFrom=%d\tTo=%d\n",pq[0],current,time);
current = time;
//We'll fill the turnaround time and wait time of that process now as it has finished
it's execution
//and the data we need is accessible right now
tat[pq[0]-1] = time - at[pq[0]-1];
wt[pq[0]-1] = tat[pq[0]-1] - bt[pq[0]-1];
//Loading the processes that arrived during the time that the first process in pq was
executing and adding those processes to the process queue for(i=0;i<n;i++)
{
```

```
if(at[i] <= time && pflag[i]== 0)
{
pq[running_processes] = i+1;
running_processes = running_processes +1;
pflag[i] = 1;
}
}
//Rearranging the process queue by pushing back all the processes and queuing the first
process to the end of the queue
rearrange_process_queue(pq,n,running_processes);
//Check if the running process has finished it's execution
if(flag == 1)
}
running_processes = running_processes -1;
printf("\n\nExecution Data:\n");
printf("-- ----------\n");
printf("|\tProcess\t|\tAT\t/\tBT\t/\tTAT\t/\tWT\t|\n"); for(i=0;i<n;i++)
{
printf(" |\t%d\t/\t%d\t/\t%d\t/\t%d\t/\t%d\t/\n",i+1,at[i],bt[i],tat[i],wt[i]);
}
printf(" -\n");
printf("\n\nAverage Waiting Time= %f\n",avg_wait_time(wt,n));
printf("Avg Turnaround Time = %f\n",avg_turnaround_time(tat,n)); return
0;
}/**
4 5
```

0 9

1 5

2 3

3 4

Input:data.txt

OUTPUT

ROUND ROBIN SCHEDULING ALGORITHM

The number of processes are set to:4

The time quantum is set to:5

The scanned process data is:

INDEX | AT BT

1 0 9

2 1 5

3 2 3

4 3 4

The order of execution:

Process queue: 1

Remaining Time: 9534

Running processes: 1

Process 1 executed for 5 bursts till time 5

Process=1 from=0 To=5

Process queue: 2 3 41

Remaining Time: 4534

Running processes: 4

Process 2 executed for 5 bursts till time 10

Process=2 From=5 To=10

Process queue: 34 1

Remaining Time: 4 034

Running processes: 3

Process 3 executed for 3 bursts till time 13

Process=3 From=10 To=13

Process queue: 41

Remaining Time: 4 004

Running processes: 2

Process 4 executed for 4 bursts till time 17

Process=4 From=13 To=17

Process queue: 1

Remaining Time: 4 000

Running processes: 1

Process 1 executed for 4 bursts till time 21

Process=1 From=17 To=21

Execution Data:

Process AT BT TAT WT

1 0 9 21 12

2 1 5 9 4

3 2 3 11 8

4 3 4 14 10

Average Waiting Time= 8.500000

Avg Turnaround Time = 13.750000

Assignment No:4 Set: A (1)

Assignment Name: Write the simulation program to implement demand paging and show the page scheduling and total number of page faults for the following given page reference string. Give input n as the number of memory frames.

Reference String: 12,15,12,18,6,8,11,12,19,12,6,8,12,15,19,8

1) Implement FIFO

```
#include<stdio.h>
int RefString[20],PT[10], nof, nor; void
Accept()
{int
i;
{
printf("Enter Reference String: \n"); for(i=0;i<nor;i++)
printf("[%d]=",i); scanf("%d",&RefString[i]);
}
}
{
int Search(int s)
int i;
}
for(i=0;i<nof; i++)
if(PT[i]==s) return(i);
return(-1);
void FIFO()
{
{
{
int i,j,k=0, Faults=0;
for(i=0;i<nor;i++)
printf("\n%2d:-",RefString[i]);
```

```
if(Search(RefString[i])==-1)
printf("\tPage Fault\t"); PT[k]=RefString[i];
for(j=0;j<nof;j++)
{
{
if(PT[j])
printf("%2d",PT[j]);
}
}
Faults++; k=(k+1)%nof;
}
}
printf("\nTotal Page Faults: %d", Faults); printf("\nHit
ratio:%d/%d", nor-Faults, nor); printf("\nMiss
ratio:%d/%d", Faults, nor);
}
void main()
printf("Enter Length of reference string: ");
scanf("%d", &nor); printf("Enter
No.of Frames: ");
scanf("%d", &nof);
Accept();
FIFO();
}/*****
Enter Length of reference string:
16 Enter No.of Frames: 3 Enter
```

Reference String:

[0]=12

[1]=15

[2]=12

[3]=18

[4]=6

[5]=8

[6]=11

[7]=12

[8]=19

[9]=12

[10]=6

[11]=8

[12]=12

[13]=15

[14]=19

[15]=8

12:- Page Fault 12

15:- Page Fault 1215

12:-

18:- Page Fault 121518

6:- Page Fault 61518

8:- Page Fault 6 818

11:- Page Fault 6 811

12:- Page Fault 12 811

19:- Page Fault 121911

12:-

OUTPUT

6:- Page Fault 1219 6

8:- Page Fault 819 6

12:- Page Fault 812 6

15:- Page Fault 81215

19:- Page Fault 191215

8:- Page Fault 19 815

Total Page Faults: 14

Hit ratio:2/16

Miss ratio:14/16

Assignment No:4 Set: A (2)

Assignment Name: Write the simulation program to implement demand paging and show the page scheduling and total number of page faults for the following given page reference string.Give input n as the number of memory frames.

Reference String: 12,15,12,18,6,8,11,12,19,12,6,8,12,15,19,8

2.Implement LRU.

#include<stdio.h> int findLRU(int

time[], int n){ int i, minimum =

time[0], pos = 0;

for(i = 1; i <n; ++i){ if(time[i]

< minimum){

minimum = time[i]; pos

=i;

}

}

}

```
return pos;
int main()
int no_of_frames, no_of_pages, frames[10], pages[30], hratio, counter = 0, time[10], flag1,
flag2, i, j, pos, faults = 0; int hit;
printf("Enter number of frames: ");
scanf("%d", &no_of_frames);
printf("Enter number of pages: ");
scanf("%d", &no_of_pages); printf("Enter
reference string: ");
for(i = 0; i < no_of_pages; ++i)
}
scanf("%d", &pages[i]);
for(i = 0; i < no_of_frames; ++i)
}
frames[i] = -1;
for(i = 0; i< no_of_pages; ++i)
{
flag1 = flag2 = 0;
for(j = 0; j< no_of_frames; ++j){ if(frames[]
== pages[i]){ counter++; time[j] = counter;
flag1 = flag2 = 1;
break;
}
}
if(flag1 == 0){ for(j = 0; j<
no_of_frames; ++j){
```

```
if(frames[j] == -1){
counter++;
faults++; frames[j] =
pages[i]; time[j] =
counter; flag2 = 1;
break;
}
}
}
}
if(flag2 == 0){ pos = findLRU(time,
no_of_frames); counter++;
faults++; frames[pos] = pages[i];
time[pos] = counter;
printf("\n"); for(j = 0; j<
no_of_frames; ++j){
printf("%d\t", frames [j]);
}
if(flag1==0)
printf("Y"); else
printf("N");
}
printf("\n\nTotal Page Faults = %d", faults);
hratio=no_of_pages-faults; printf("\n
Hitratio:%d/%d",hratio,no_of_pages); printf("\n
Missratio:%d/%d",faults,no_of_pages);
```

```
return 0;

}
```

QUTPUT

```
Enter number of frames: 3

Enter number of pages: 16

Enter reference string: 12 15 12 18 6 8 11 12 19 12 6 8 12 15 19 8

12 -1 -1 Y

12 15 -1 Y

12 15 -1 N

12 15 18 Y

12 6 18 Y

8 6 18 Y

8 6 11 Y

8 12 11 Y

19 12 11 Y

19 12 11 N

19 12 6 Y

8 12 6 Y

8 12 6 N

8 12 15 Y

19 12 15 Y

19 8 15 Y

Total Page Faults = 13

Hitratio:3/16

Missratio:13/16
```

Assignment No:4 Set: B (1)

Assignment Name: Write the simulation program to implement demand paging and show the page scheduling and total number of page faults for the following given page reference string.Give input n as the number of memory frames.

Reference String: 12,15,12,18,6,8,11,12,19,12,6,8,12,15,19,8 1)

Implement OPT.

****:

```
#include<stdio.h>

int main()

int reference_string[25], frames[25], interval[25];

int pages, total_frames, page_faults = 0; int m, n,

temp, flag,hr,found;

int position, maximum_interval, previous_frame = -1;

printf("\nEnter Total Number of Pages:\t"); scanf("%d",

&pages);

printf("\nEnter Values of Reference String\n");

for(m = 0; m < pages; m++)

{

printf("Value No.[%d]:\t", m + 1);

scanf("%d", &reference_string[m]);

}

printf("\nEnter Total Number of Frames:\t");

scanf("%d", &total_frames);

for(m = 0; m < total_frames; m++)

frames[m] = -1;

}

for(m = 0; m < pages; m++)

{
```

```
flag = 0;
for(n = 0; n < total_frames; n++)
{
flag = 1;
if(frames[n] == reference_string[m])
{
printf("\t");
break;
}
}
if(flag == 0)
{
if (previous_frame == total_frames - 1)
{
for(n = 0; n < total_frames; n++)
{
for(temp = m + 1; temp < pages; temp++)
}
interval[n] = 0;
if (frames[n] == reference_string[temp])
interval[n] = temp - m;
break;
}
}
} found =
0;
```

```
for(n = 0; n < total_frames; n++)
{
if(interval[n] == 0)
{
position = n;
found = 1;
break;
}
{
else
position = ++previous_frame;
found = 1;
}
if(found == 0)
{
maximum_interval = interval[0];
position = 0;
for(n = 1; n < total_frames; n++)
{
if(maximum_interval < interval[n])
}
}
}
maximum_interval = interval[n];
position = n;
frames[position] = reference_string[m];
```

```
printf("FAULT\t");
page_faults++;
}
for(n = 0; n < total_frames; n++)
{
if(frames[n]!= -1)
printf("%d\t", frames[n]);
}
}
}
printf("\n");
printf("\nTotal Number of Page Faults:\t%d\n", page_faults);
hr=pages-page_faults; printf("\nHit Ratio: %d/%d",hr,pages);
printf("\nMiss Ratio: %d/%d",page_faults, pages);
return 0;
}
```

OUTPUT

```
Enter Total Number of Pages: 16
Enter Values of Reference String
Value No.[1]: 12
Value No.[2]: 15
Value No.[3]: 12
Value No.[4]: 18
Value No. [5]: 6
Value No.[6]: 8
Value No.[7]: 11
```

Value No.[8]: 12

Value No.[9]: 19

Value No. [10]: 12

Value No.[11]: 6

Value No. [12]: 8

Value No.[13]: 12

Value No. [14]: 15

Value No. [15]: 19

Value No.[16]: 8

Enter Total Number of Frames: 4

FAULT 12

FAULT 12 15

12 15

FAULT 12 15 18

FAULT 12 15 18 6

FAULT 12 15 8 6

FAULT 12 11 8 6

12 11 8 6

FAULT 12 19 8 6

12 19 8 6

12 19 8 6

12 19 8 6

12 19 8 6

FAULT 15 19 8 6

15 19 8 6

15 19 8 6

Total Number of Page Faults: 8

Hit Ratio: 8/16

Miss Ratio: 8/16

Assignment No:4 Set: B (2)

Assignment Name: Write the simulation program to implement demand paging and show the page scheduling and total number of page faults for the following given page referencestring.Give input n as the number of memory frames.

Reference String: 12,15,12,18,6,8,11,12,19,12,6,8,12,15,19,8

2) Implement MFU

```
#include<stdio.h>

int RefString[20],PT[10], nof, nor,hr; void

Accept()

{ int

i;

printf("Enter Reference String: \n");

for(i=0;i<nor;i++)

printf("[%d]=",i);

scanf("%d",&RefString[i]);

}

}

int Search(int s)

{

int i;

for(i=0;i<nof; i++)

if(PT[i]==s)

return(i);

return(-1);
```

```
}
{
int GetMFU(int e, int s)
int i,j,cnt1=0,cnt2,posi;
i=s;
do
cnt2=0;
for(j=e-1;j>=0;j--)
{
if(PT[i]==RefString[j])
{
cnt2++;
}
}
if(cnt2>cnt1)
cnt1=cnt2;
posi = i;
}
i=(i+1)%nof;
}while(i!=s);
return(posi);
}
void MFU()
{ int
i,j,k,Faults=0;
for(k=0,i=0; k<nof && i<nor; i++)
```

```
printf("\n%2d\t",RefString[i]);
if(Search(RefString[i])==-1)
{
PT[k]=RefString[i];
for(j=0;j<nof;j++)
{
if(PT[j])
printf("%2d\t",PT[j]);
}
Faults++;
k++;
}
}
k=0;
while(i<nor)
printf("\n%2d\t",RefString[i]);
if(Search(RefString[i])==-1)
k = GetMFU(i,k);
PT[k]=RefString[i]; k=(k+1)%nof;
for(j=0;j<nof;j++)
if(PT[j])
printf("%2d\t",PT[j]); }
Faults++;
}
i++;
}
```

```
printf("\nTotal Page Faults:
%d", Faults); hr=nor-(Faults);
printf("\nHit Ratio: %d/%d",hr,nor);
printf("\nMiss Ratio: %d/%d",Faults, nor);
}
void main()
{
printf("Enter Length of reference string: ");
scanf("%d",&nor);
printf("Enter No.of Frames: ");
scanf("%d",&nof);
Accept();
MFU();
```

OUTPUT

```
Enter Length of reference string:
16 Enter No.of Frames: 4 Enter
Reference String:
[0]=12
[1]=15
[2]=12
[3]=18
[4]=6
[5]=8
[6]=11
[7]=12
[8]=19
```

[9]=12

[10]=6

[11]=8

[12]=12

[13]=15

[14]=19

[15]=8

12 12

15 12 15

12

18 12 15 18

6 12 15 18 6

8 8 15 18 6

11 8 11 18 6

12 8 11 12 6

19 8 11 19 6

12 8 11 19 12

6 8 11 19 6

8

12 12 11 19 6

15 15 11 19 6

19

8 15 11 8 6

Total Page Faults: 13

Hit Ratio: 3/16

Miss Ratio: 13/16

Assignment No:4 Set: C (1)

Assignment Name: Write the simulation program to implement demand paging and show the page scheduling and total number of page faults for the following given page reference string. Give input n as the number of memory frames.

Reference String: 2,5,2,8,5,4,1,2,3,2,6,1,2,5,9,8

1) Implement MRU

```
#include<stdio.h>

void main()

{

int f[3]={-1,-1,-1),cnt[3]={-1,-1,-1},a[50],i,j,no,pfault=0,hratio;

printf("\n++++++++++++++++++ Most Recently Used Page Algo

++++++++++++++++++\n");

printf("-

printf("\nEnter the no of

pages:"); scanf("%d",&no);

printf("\nEnter the Pages:");

for(i=0;i<no;i++)

scanf("%d",&a[i]);

printf("\nPageNo\tFrame\t\tResult");

for(i=0;i<no;i++)

{

printf("\n%d",a[i]);

if(f[0]!=a[i] &&f[1]!=a[i] &&f[2]!=a[i])

{

{

pfault++; if(f[0]==-1)

f[0]=a[i];
```

```
cnt[0]=i;
}
else if(f[1]==-1)
{
}
f[1]=a[i];
{
cnt[1]=i;
else if(f[2]==-1)
f[2]=a[i]; cnt[2]=i;
");
}
else
{
}
if(cnt[0]>cnt[1] && cnt[0]>cnt[2])
f[0]=a[i];
{
}
{
}
cnt[0]=i;
else
if(cnt[1]>cnt[0] && cnt[1]>cnt[2])
f[1]=a[i]; cnt[1]=i;
else
```

```
f[2]=a[i]; cnt[2]=i;
}
}
printf("\t%d %d %d\t\tPage Fault",f[O],f[1],f[2]);
else
{
for(j=0;j<3;j++)
if(f[j]==a[i]) cnt[j]=i;
printf("\t%d %d %d\t\tNo Page Fault....",f[0],f[1],f[2]);
}
}
}
printf("\nTotal no of Page
Faults:%d",pfault); hratio=no-pfault;
printf("\n Hitratio:%d/%d", hratio,no);
printf("\n Missratio:%d/%d",pfault, no);
```

OUTPUT

```
++++++++++++++++++ Most Recently Used Page Algo ++++++++++++++++++
Enter the no of pages:16
Enter the Pages:2
5
2
8
5
4
1
```

2

3

2

6

1

2

5

9

8

PageNo Frame Result

2 2-1-1 Page Fault

5 25-1 Page Fault

2 25-1 No Page Fault....

8 258 Page Fault

5 258 No Page Fault....

4 248 Page Fault

1 218 Page Fault

2 218 No Page Fault....

3 318 Page Fault

2 218 Page Fault

6 618 Page Fault

1 618 No Page Fault....

2 628 Page Fault

5 658 Page Fault

9 698 Page Fault

8 698 No Page Fault....

Total no of Page Faults:11

Hitratio:5/16

Missratio:11/16

Assignment No:4 Set: C (2)

Assignment Name: Write the simulation program to implement demand paging and show the page scheduling and total number of page faults for the following given page reference string.Give input n as the number of memory frames.

Reference String: 2,5,2,8,5,4,1,2,3,2,6,1,2,5,9,8

2)Implement Second Chance Page Replacement.

****

#include<stdio.h> #define

SIZE 3

int full=0;//To check whether all frames are filled int

a[21];//To take the input

int ref[SIZE];

char r=1; int

frame [SIZE];

unsigned char

addref[SIZE]={0}; int repptr=0;

int count=0; int display()

{int i;

printf("\nThe elements in the frame are\n"); for(i=0;i<full;i++)

printf("%d\n",frame[i]);

}

//For showing bits

int bitrep(char f)//Program for showing the bits

{char p,fl; int j;

```
char i=1;
for(j=7;j>=0;j--)
}
{p=i<<j; fl=p&f;
if(fl==0)
printf("0");
else
printf("1");
printf("\n");
}
//For showing bit wise int
additionalreference()
{int i,min;
min=0;
for(i=1;i<SIZE;i++)
if(addref[min]>addref[i])//Finding the smallest decimal value from all the strings
available min=i; repptr=min;
return repptr;
{
}
int Pagerep(int ele)
int temp;
repptr=additionalreference();
temp=frame[repptr];
frame[repptr]=ele; ref[repptr]=1;
}
```

```
return temp;
int Pagefault(int ele)
}
{if(full!=SIZE) {ref[full]=1;
frame[full++]=ele;
else
}
printf("The page replaced is %d",Pagerep(ele));
int Search(int ele)
{int i,flag;
flag=0;
{
if(full!=0)
for(i=0;i<full;i++)
if(ele==frame[i]) {
flag=1;ref[i]=1;
}
break; }}
return flag;
int main()
{int n,i,k;
r=r<<7;
bitrep(r);
FILE *fp;
fp=fopen("Input.txt","r");
printf("The number of elements in the reference string are
```

```
:"); fscanf(fp,"%d",&n); printf("%d",n); for(i=0;i<n;i++)
fscanf(fp,"%d",&a[i]);
printf("\nThe elements present in the string are\n");
for(i=0;i<n;i++) printf("%d ",a[i]); printf("\n\n");
for(i=0;i<n;i++)
{
if(Search(a[i])!=1)
{Pagefault(a[i]);
display();
count++;
}
printf("\nThe values in shift registers related to three frames are\n");
//Shifing of registers
for(k=0;k<SIZE;k++)
{
addref[k]=addref[k]>>1;//Shifting all the shift registers by one bit
if(ref[k]==1)
{ addref[k]=(r|addref[k]);//Bit OR operation for shifting
ref[k]=0;
}
// bitrep(addref[k]);
}
//End of shifting
}
printf("\nThe number of page faults are %d\n",count);
return 0;
```

}

OUTPUT

The number of elements in the reference string are :16

The elements present in the string are

2528541232612598

The elements in the frame are

2

The elements in the frame are

2

5

The elements in the frame are

2

5

8

The page replaced is 8

The elements in the frame are

2

5

4

The page replaced is 2

The elements in the frame are

1

5

4

The page replaced is 5

The elements in the frame are

1

2

4

The page replaced is 4

The elements in the frame are

1

2

3

The page replaced is 1

The elements in the frame are

6

2

3

The page replaced is 3

The elements in the frame are

6

2

1

The page replaced is 6

The elements in the frame are

5

2

1

The page replaced is 1

The elements in the frame are

5

2

9

The page replaced is 5

The elements in the frame are

8

2

9

The number of page faults are 12

Assignment No:4 Set: C (3)

Assignment Name: Write the simulation program to implement demand paging and show the page scheduling and total number of page faults for the following given page reference string.Give input n as the number of memory frames.

Reference String: 2,5,2,8,5,4,1,2,3,2,6,1,2,5,9,8 3)Least

Frequently Used(LFU).

```
#include<stdio.h>

int main()

{

int total_frames, total_pages, hit = 0; int

pages[25], frame[10], arr[25], time[25]; int m,

n, page, flag, k, minimum_time,temp,hr;

printf("Enter Total Number of Pages:\t");

scanf("%d", &total_pages);

printf("Enter Total Number of Frames:\t");

scanf("%d", &total_frames);

for(m = 0; m < total_frames; m++)

{

frame[m] = -1;
```

```
}
for(m = 0; m < 25; m++)
{
arr[m] = 0;
}
printf("Enter Values of Reference String\n");
for(m = 0; m < total_pages; m++)
{
printf("Enter Value No.[%d]:\t", m + 1);
scanf("%d", &pages[m]);
}
{
printf("\n");
for(m = 0; m < total_pages; m++)
arr[pages[m]]++;
time[pages[m]] =
m; flag = 1;
= frame[0];
k
for(n = 0; n < total_frames; n++)
if(frame[n] == -1 || frame[n] == pages[m])
{
if(frame[n] != -1)
{
hit++;
} flag =
```

```
0; frame[n] =
pages[m];
break;
}
}
if(flag)
{
if(arr[k] > arr[frame[n]])
{
k = frame[n];
minimum_time = 25;
for(n = 0; n< total_frames; n++)
{
if(arr[frame[n]] == arr[k] && time[frame[n]] < minimum_time)
{
temp = n;
minimum_time = time[frame[n]];
}
arr[frame[temp]] = 0;
frame[temp] = pages[m];
}
for(n = 0; n < total_frames; n++)
{
printf("%d\t", frame[n]);
}
printf("\n");
```

```
}
printf("\nPage Fault:\t%d\n", total_pages-hit);
hr=total_pages-(total_pages-hit); printf("\nHit Ratio:
%d/%d",hr,total_pages); printf("\nMiss Ratio:
%d/%d",total_pages-hit,total_pages); return 0;
}/***************OUTPUT****
Enter Total Number of Pages: 16
Enter Total Number of Frames: 3
Enter Values of Reference String
Enter Value No.[1]: 2
Enter Value No.[2]: 5
Enter Value No.[3]: 2
Enter Value No.[4]: 8
Enter Value No.[5]: 5
Enter Value No.[6]: 4
Enter Value No.[7]: 1
Enter Value No.[8]: 2
Enter Value No.[9]: 3
Enter Value No.[10]: 2
Enter Value No.[11]: 6
Enter Value No.[12]: 1
Enter Value No.[13]: 2
Enter Value No.[14]: 5
Enter Value No.[15]: 9
Enter Value No.[16]:8
2 -1 -1
```

2 5 -1

2 5 -1

2 5 8

2 5 8

2 5 4

2 5 1

2 5 1

2 5 3

2 5 3

2 5 6

2 5 1

2 5 1

2 5 1

2 5 9

2 5 8

Page Fault: 10

Hit Ratio: 6/16

Miss Ratio: 10/16